Highlights

MPC using an on-line TS fuzzy learning approach with application to autonomous driving

Eugenio Alcalá, Iury Bessa, Vicenç Puig, Olivier Sename, Reinaldo Palhares

- We propose a novel learning control framework based on model predictive control (MPC) and moving horizon estimator (MHE)
- We use evolving Takagi-Sugeno fuzzy ellipsoidal information granules (TS-EEFIGs) for learning nonlinear dynamics
- The MPC and MHE framework is driven by TS-EEFIGs learned models
- The integrated approach with MPC, MHE, and TS-EEFIG is used for solving an autonomous driving problem in a well-known case study based on a scale vehicle.

MPC using an on-line TS fuzzy learning approach with application to autonomous driving

Eugenio Alcalá[a], Iury Bessa[b,c], Vicenç Puig[a], Olivier Sename[d], Reinaldo Palhares[e]

[a] *Supervision, Safety and Automatic Control Research Center (CS2AC) of the Universitat Politécnica de Catalunya, Rambla Sant Nebridi, 22, Barcelona, 08222, Spain*
[b] *Department of Electricity, Federal University of Amazonas, Av. Gal. Rodrigo Octavio Jordão Ramos, 1200, Manaus, 69067-005, Amazonas, Brasil*
[c] *Graduate Program in Electrical Engineering, Federal University of Minas Gerais, Av. Pres. Antônio Carlos, 6627, Belo Horizonte, 31270-901, Minas Gerais, Brasil*
[d] *Control System Department at GIPSA-lab, 11 rue des Mathématiques, Grenoble, 38400, France*
[e] *Department of Electronics Engineering, Federal University of Minas Gerais, Av. Pres. Antônio Carlos, 6627, Belo Horizonte, 31270-901, Minas Gerais, Brasil*

Abstract

The control of complex nonlinear systems (such as autonomous vehicles) usually requires mathematical tools that depend on models that are eventually not available with enough accuracy. In this paper, a novel data-driven Model Predictive Control (MPC) framework is proposed based on a data-driven approach to learn Takagi-Sugeno (TS) fuzzy models for nonlinear systems. To address the data TS modeling, we use the Evolving TS Fuzzy Ellipsoidal Information Granules (TS-EEFIG) approach to obtain a polytopic representation as well as a set of membership functions that allows to use efficient linear control tools to handle complex nonlinear systems. In particular, the formulated approach is applied for the autonomous driving control problem of a racing vehicle in real-time. The proposed control uses references provided by an external trajectory planner offering a high driving performance in racing mode. The control-estimation scheme is tested in a simulated racing environment to show the potential of the proposed approach.

Keywords: Takagi-Sugeno fuzzy models, Model predictive control, Evolving systems, Granular Computing, Learning-based control

1. Introduction

Model predictive control (MPC) has been proved as powerful tool for the control of autonomous vehicles, both ground and aerial (see as e.g. [1, 2, 3] for some recent references). The main reason is because MPC allows to handle input and state constraints as well as nonlinearities introduced by the vehicle dynamics. Moreover, MPC is able to deal with constraints, such as collision or track constraints, and provides an intuitive way for trading off competing goals by conveniently defining the cost function. However, in most of the MPC approaches, a system model is required to adequately capture the vehicle dynamics while being simple enough to be used in an online optimization framework.

As discussed in [4], autonomous system models provide a relationship between physical inputs and the states of the system. But, choosing the right system model is often an art and is highly dependent on the task complexity as well as the availability of computational power. Using a complex model may render MPC computationally intractable, whereas an overly simple model can result in reduced performance or even collisions. In addition, the dynamics can change between different scenarios, imposing the need to adapt the system model during operation. For all these reasons, there is a growing interest in developing data-based and learning-based approaches coupled with MPC, as discussed in [4] and in [5]. In [6, 7], a Lyapunov-based MPC scheme for non-linear systems is proposed using a recurrent neural-network as the prediction model. This approach is further extended to include on-line learning [8].

In recent years, the amount of learning-based MPC approaches has increased immensely. Particularly, in the autonomous driving field, we have witnessed new approaches as the end to end driving where the goal consists on guiding the vehicle by means of using learning algorithms and input sensors data. In [9], a deep reinforcement learning framework is proposed that takes raw sensor measurements as inputs and outputs driving actions. Nowadays, from a control point of view, several strategies are starting to use learning tools to improve their capabilities while guaranteeing overall system stability. We have witnessed an advance in this field reaching learning techniques to adjust controllers, identify some parameters inside models or even control nonlinear systems. In [10, 11], some solutions for controlling the longitudinal velocity of a car based on learning human behaviour are presented. In [12], Gaussian process regression is used for online learning of race car dynamics and for autonomous driving based on an MPC framework. In [13, 14, 15], it is proposed a reference-free iterative MPC strategy able to learn from previous laps information. Similarly, in [16], an MPC framework based on information theoretic interpretation is proposed for the optimal control problem using hybrid-physics and pure machine learning and applied to the problem of autonomous driving.

Most of the approaches listed above are based on learning some policies to drive the vehicle independently of a physical model. In this work, we are interested on learning a realistic and accurate representation of the system dynamics to improve the control performance. In [17], an evolving neuro-fuzzy controller so-called parsimonious controller is used for attitude and altitude control of micro aerial vehicles. In [12], a simple initial vehicle model is used and it is enhanced on-line by learning the model error using Gaussian process regression and measured vehicle data.

In this paper, we propose the use of of evolving ellipsoidal fuzzy information granules with Takagi-Sugeno consequent (TS-EEFIG) described in [18] for learning TS fuzzy representations for nonlinear systems from input-output data. Evolving models as TS-EEFIG are universal approximators as shown in [19, 20] and are effectively employed for identification [21], estimation [20], prediction [22], forecasting [23], classification [24], fault diagnosis [25], fault prognostics [26], and control [27] problems. In particular, this paper proposes to use the TS-EEFIGs within an MPC framework fed by a moving horizon estimator (MHE) for ensuring the reference tracking of the nonlinear system approximated by TS-EEFIG model. The TS-EEFIG offers the advantages of easily handling nonlinear and multidimensional systems due to the ellipsoidal membership functions and providing the versatile TS consequent structure that allows to use most of the well known control techniques. In [28], a similar framework with MPC and MHE is proposed for ANFIS models identified offline without the online learning ability. The proposed learning-based MPC approach requires some historical data to start the model, but it is able to learn and improve this model when it is used. It allows the use of the proposed model in scenarios never experimented before. Compared to the classical MPC approaches [1, 3, 2], the proposed approach disregards the knowledge of the physical model and parameters. In particular, for the autonomous driving application, the estimation of some parameters is a tough task which is avoided by means of the proposed TS-EEFIG learning algorithm.

The main contribution of this work is to introduce novel data-driven MPC and MHE strategies based on evolving TS fuzzy models (TS-EEFIGs) accurately learned from both historical and online data. Finally, the MPC and MHE based on TS-EEFIG is used for providing predictive optimal solution for an autonomous driving problem. The experimental evaluation indicates the learning ability of the TS-EEFIG models and the efficacy when applied to the autonomous driving problem. In short, the main contributions of this work are listed as follows:

- an integrated framework of MPC and MHE driven by TS fuzzy models is presented;
- the TS-EEFIG is used to learn effective TS fuzzy representations of nonlinear systems from input-output data;

- the learning ability of EEFIG is used to provide a novel MPC approach that is integrated to the MHE and both are based on an evolving TS fuzzy model that is able to improve the MPC performance when presented to different dynamical scenarios;
- The integrated approach with the MPC, MHE, and TS-EEFIG is used for solving an autonomous driving problem in a well-known case study based on a scale vehicle [2] .

The remaining of this paper is structured as follows. Section 2 presents the proposed TS-EEFIG learning-based method and its main components. Section 3 describes the MPC and MHE approaches that uses TS fuzzy models for dealing with control and estimation problems. Section 4 presents the integration of the MPC and MHE framework with the online learning methodology for TS-EEFIG models. Section 5 introduces its application to a autonomous vehicle case study to assess the methodology, as well as various performance results. Finally, Section 6 presents the conclusions and future work.

2. Learning the TS model

Data-driven fuzzy models are effective tools for obtaining mathematical representations useful for tasks as estimation and control but that also conserves some interpretability. In particular, the granular computing and granular fuzzy models provide a framework that is able to formulate, from data, meaningful concepts for abstract data clusters [29].

In this paper, TS fuzzy models are obtained by means of Evolving Ellipsoidal Fuzzy Information Granules (EEFIG) and the Principle of Justifiable Granularity (PJG) applied to data streams, as proposed in [18].

2.1. Granular TS Fuzzy Ellipsoidal Models

Consider a nonlinear dynamic discrete-time system described as follows

$$x_{k+1} = f(x_k, u_k, k) \tag{1}$$

where $x_k \in \mathbb{R}^{n_x}$ and $u_k \in \mathbb{R}^{n_u}$ are, respectively, the state and input vectors, and the nonlinear map $f : \mathbb{R}^{n_x} \times \mathbb{R}^{n_u} \mapsto \mathbb{R}^{n_x}$ is unknown. In this paper, it is obtained a TS fuzzy model based on the granulation of the state and input variables that approximates f. For this purpose, the vector of premise variables is defined as

$\zeta_k = \left[x_k^\top \ u_k^\top\right]^\top$, with $\zeta_k \in \mathbb{R}^{n_\zeta}$, $n_\zeta = n_x + n_u$. Thus, the granular fuzzy rule-based models obtained are of the form

$$\begin{aligned} \mathcal{R}^i : &\text{IF} \quad \zeta_k \quad \text{is} \quad \mathcal{G}_k^i \\ &\text{THEN} \quad x_{k+1}^i = A_k^i x_k + B_k^i u_k \end{aligned} \tag{2}$$

where A_k^i and B_k^i are matrices with appropriate dimensions and time-varying coefficients, $\mathcal{R}^i$ is the i-th rule related to the i-th time-varying information granule $\mathcal{G}_k^i$, with $i \in \mathbb{N}_{\leq N_G}$, and N_G is the number of rules granules, that is equal to the number of granules, that is also time-varying.

An information granule $\mathcal{G}_k^i$ is a fuzzy set obtained by means of the application of the PJG for a dataset $\mathcal{Z}_k = \{\zeta_1, \zeta_2, \ldots, \zeta_k\}$. The PJG should define the model granulation by solving a multi-objective optimization problem for obtaining a trade-off between the coverage (experimental evidence) and specificity (semantic meaning). These granule properties are quantified by means of the granule mean $\nu_k^i \in \mathbb{R}^{n_\zeta}$, and its coverage bounds $\underline{\zeta}_k^i \in \mathbb{R}^{n_\zeta}$ and $\overline{\zeta}_k^i \in \mathbb{R}^{n_\zeta}$. In particular, in this work is used the concept of ellipsoidal fuzzy granules described in [30] that eases the computation of membership degrees for multidimensional data. The ellipsoidal granular Takagi-Sugeno fuzzy rule-based model (2) used in this paper is simply called TS-EEFIG.

In this sense, consider a hyper-ellipsoid $\mathcal{E}_k^i$ centered at ν_k^i with a covariance matrix Σ_k^i, with the following formulation:

$$\mathcal{E}_k^i = \{\zeta_k \in \mathcal{Z}_k \mid M_k^i \leq \epsilon\}, \tag{3}$$

where $M_k^i = (\zeta_k - \nu_k^i)^\top {\Sigma_k^i}^{-1} (\zeta_k - \nu_k^i)$ is the Mahalanobis distance from ζ_k to ν_k^i, and ϵ is a positive threshold defined as the inverse of χ^2 distribution with n_ζ-degrees of freedom. The condition defined in (3) means that a given data sample is inserted in the ellipsoidal set $\mathcal{E}_k^i$ considering their limits based on the covariance Σ_k^i and the center ν_k^i that is set as mean of the data samples in $\mathcal{E}_k^i$. The data samples admitted in $\mathcal{E}_k^i$ can be partitioned into subsets according to the sample data position with respect to the mean ν_k^i:

$$\mathcal{Z}_{k,j}^- = \left\{\zeta_{k,j} \in \mathcal{E}_k^i \mid \zeta_{k,j} < \nu_{k,j}^i\right\}, \tag{4}$$

$$\mathcal{Z}_{k,j}^+ = \left\{\zeta_{k,j} \in \mathcal{E}_{k,j}^i \mid \zeta_{k,j} \geq \nu_{k,j}^i\right\}, \tag{5}$$

where $\zeta_{k,j}$ and $\nu_{k,j}^i$ denote the j-th element of the respective vectors ζ_k and ν_k^i, with $j \in \mathbb{N}_{\leq n_\zeta}$. The dataset partitions into subsets $\mathcal{Z}_{k,j}^+$ and $\mathcal{Z}_{k,j}^+$ allow to obtain the lower and upper optimal bounds $\underline{\zeta}_k^i$ and $\overline{\zeta}_k^i$. For this purpose, consider that $S_{k,j}^i$ and $T_{k,j}^i$ are, respectively, the averages of $\mathcal{Z}_{k,j}^+$ and $\mathcal{Z}_{k,j}^+$, thus, according to [18]

$$\underline{\zeta}^i_{k,s} = \max\{\nu^i_{k,s} - \lambda S^i_{k,j,s}, \beta_S\} + (\nu^i_{k,s} - \max\{\nu^i_{k,s} - \lambda S^i_{k,j,s}, \beta_S\})\iota \tag{6}$$

$$\overline{\zeta}^i_{k,s} = \min\{\nu^i_{k,s} + \lambda T^i_{k,j,s}, \beta_T\} - (\min\{\nu^i_{k,s} + \lambda T^i_{k,j,s}, \beta_T\} - \nu^i_{k,s})\iota, \tag{7}$$

where $[\beta_S, \beta_T) = [0, +\infty)$, $\lambda \in [2, 4]$ and $\iota \in [0, 1]$ as suggested in [30], and $S^i_{k,j,s}$, $T^i_{k,j,s}$, $\underline{\zeta}^i_{k,s}$ and $\overline{\zeta}^i_{k,s}$ are the s-th element of the vectors $S^i_{k,j}$, $T^i_{k,j}$, $\underline{\zeta}^i_k$ and $\overline{\zeta}^i_k$ respectively.

All the above-mentioned time-varying variables are updated at each instant k in order to learn the f mapping dynamics. Thus, the approximation of (1) is provided by means of the following equation

$$x_{k+1} = \sum_{i=1}^{N_G} g^i_k(\zeta_k)\, x^i_{k+1} \tag{8}$$

where $g^i_k : \mathbb{R}^{n_\zeta} \mapsto [0, 1]$ is the normalised ellipsoidal membership degree computed as

$$g^i_k(\zeta_k) = \frac{\xi^i_k(\zeta_k)}{\sum_{i=1}^{N_G} \xi^i_k(\zeta_k)} \tag{9}$$

and membership degree $\xi^i_k(\cdot)$ for the k-th premise variables sample is given by

$$\xi^i_k(\zeta_k) = \exp\left(-2 \quad \sqrt{\sum_{j=1}^{n_\zeta} \left(\frac{\zeta_{k,j} - \nu^i_{k,j}}{\overline{z}^i_{k,j} - \underline{z}^i_{k,j}}\right)^2}\right). \tag{10}$$

Alternatively, (8) can be represented as follows

$$x_{k+1} = \bar{A}_k(\zeta_k)\, x_k + \bar{B}_k(\zeta_k)\, u_k \tag{11}$$

where

$$\bar{A}_k(\zeta_k) = \sum_{i=1}^{N_G} g^i_k(\zeta_k)\, A^i_k, \tag{12}$$

$$\bar{B}_k(\zeta_k) = \sum_{i=1}^{N_G} g^i_k(\zeta_k)\, B^i_k. \tag{13}$$

2.2. Updating the TS Fuzzy Ellipsoidal Information Granules

The update procedures of the antecedent of the granular fuzzy model (2), i.e., the creation of new granules and update of the membership functions (9), are described in detail in [18]. The general steps of antecedent updates are summarised in this subsection.

The first step of the granule update is to check the admission condition (3) for the new data sample ζ_k. Only admitted samples are used for updating a granule. However, an admitted sample is used for update the granules properties only if it is able to improve the granule performance index $\mathcal{I}\left(\mathcal{G}_k^i, \zeta_k\right)$ that is a cost function related to the PJG which is a trade-off between the coverage and specificity of the granule. The granule index for a given data sample is computed as follows

$$\mathcal{I}\left(\mathcal{G}_k^i, \zeta_k\right) = M_k^i \sum_{\zeta_k \in \mathcal{E}_k^i} g_k^i\left(\zeta_k\right) \tag{14}$$

The iterative optimization procedure for (14) is detailed in [18]. A data sample ζ_k is said to improve the performance index of $\mathcal{G}_{k-1}^i$ if $\mathcal{I}\left(\mathcal{G}_k^i, \zeta_k\right) \geq \mathcal{I}\left(\mathcal{G}_{k-1}^i, \zeta_k\right)$.

Thus, assuming that $\zeta_k \in \mathcal{E}_k^i$ and that ζ_k improves the performance index of $\mathcal{G}_{k-1}^i$, the following equations are used to compute $\mathcal{G}_k^i$ and its properties given ζ_k and $\mathcal{G}_{k-1}^i$

$$\nu_k^i = \nu_{k-1}^i + g_{k-1}^i\left(\zeta_k\right)\left(\zeta_k - \nu_{k-1}^i\right), \tag{15}$$

$$S_{k,j,l}^i = \frac{\left(s_{k-1,j} S_{k-1,j,l}^i\right) + \left(\nu_{k,l}^i - \zeta_{k,l}^i\right)}{s_{k,j}}, \tag{16}$$

$$T_{k,j,l}^i = \frac{\left(t_{k-1,j} T_{k-1,j,l}^i\right) + \left(\zeta_{k,l}^i - \nu_{k,l}^i\right)}{t_{k,j}}, \tag{17}$$

for all $\{j, s\} \subset \mathbb{N}_{\leq n_\zeta}$, where $s_{k,j}$ and $t_{k,k}$ denotes the number of samples that are within, respectively, $\mathcal{Z}_{k,j}^+$ and $\mathcal{Z}_{k,j}^-$ until the instant k. Thus, the coverage bounds ($\underline{\zeta}_k^i$ and $\overline{\zeta}_k^i$) and the membership degree functions (g_k^i and ξ_k^i) can be updated by substituting the results of (15)–(17) in (6)–(7) and (9)–(10), respectively. The covariance matrix Σ_k^i is updated as follows.

$$\Sigma_k^{i\,-1} = \Gamma_k^i \left[\Sigma_{k-1}^{i\,-1} - \frac{\Sigma_{k-1}^{i\,-1}\left(\zeta_k - \nu_k^i\right)^\top\left(\zeta_k - \nu_k^i\right)\Sigma_{k-1}^{i\,-1}}{\Lambda_k^i + \left(\zeta_k - \nu_k^i\right)^\top \Sigma_{k-1}^{i\,-1}\left(\zeta_k - \nu_k^i\right)}\right], \tag{18}$$

$$\tau_k^i = \sum_j \xi_{\mathbb{J}}^i\left(\zeta_j\right)\ , \quad \rho_k^i = \sum_{j\in\mathbb{J}}\left(\xi_j^i\left(\zeta_j\right)\right)^2\ , \quad \mathbb{J} = \left\{j \in \mathbb{N}_{\leq k} | \zeta_j \in \mathcal{E}_j^i\right\},$$

$$\Gamma_k^i = \frac{\rho_{k-1}^i \left({\rho_k^i}^2 - \tau_{k-1}^i \right)}{\rho_k^i \left({\rho_{k-1}^i}^2 - \tau_{k-1}^i \right)} \ , \quad \Lambda_k^i = \frac{\rho_k^i \left({\rho_{k-1}^i}^2 - \tau_{k-1}^i \right)}{\rho_{k-1}^i \xi_{k-1}^i \left(\zeta_k \right) \left(\rho_k^i + \xi_k^i \left(\zeta_k \right) - 2 \right)}.$$

As far as k is increasing, i.e., incoming of new data samples, new granules should be created. For this purpose, a memory horizon is established with φ samples. This memory is used to build an auxiliary granule $\mathcal{G}_k^{\text{aux}}$ that is incorporated to the granular fuzzy model (2) only if two necessary conditions are met:

- The $\mathcal{G}_k^{\text{aux}}$ meets the c-separation condition [31] with respect to all the other granules $\mathcal{G}_k^i$ for $i \in \mathbb{N}_{\leq N_G}$. The granule $\mathcal{G}_k^{\text{aux}}$ is c-separable of $\mathcal{G}_k^i$ if and only if the following condition is met.

$$||\nu_k^{\text{aux}} - \nu_k^i|| \geq c\sqrt{n_\zeta \max \left(\overline{\sigma} \left(\Sigma_k^{\text{aux}} \right), \overline{\sigma} \left(\Sigma_k^{\text{i}} \right) \right)}, \tag{19}$$

  where Σ_k^{aux} and ν_k^{aux} denote respectively the covariance and mean of $\mathcal{G}_k^{\text{aux}}$, $\overline{\sigma}(C)$ denotes the largest eigenvalue of C, and $c \in [0, \infty)$ specifies the minimum required separation level between two information granules.

- Defining an anomaly as a data sample $\zeta_k \notin \mathcal{E}_k^i$ for $i \in \mathbb{N}_{\leq N_G}$ (i.e., it does not belong to any of the existing granules), thus, the number of anomalies n_a within the set of last φ samples is such that $n_a > \bar{n}_a$, where $\bar{n}_a$ is an arbitrary number of maximum admissible anomalies.

The consequent parameters of (2), i.e., the matrices A_k^i and B_k^i have to be re-estimated every time the i-th granule is activated, what occurs when the i-th data sample is admitted in $\mathcal{E}_k^i$ according to (3). In this work, the parameters are estimated by using batch windowed least squares (WLS) for a moving window, when the model is trained offline, and by using recursive least squares (RLS) for online learning.

2.3. Estimating the consequent parameters of TS Fuzzy Ellipsoidal Information Granules

The TS-EEFIG can be obtained from a batch dataset from (1), using the evolving granulation procedures only as a system identification algorithm, or it can be updated on-line in order to perform the TS-MPC and TS-MHE based on an on-line fuzzy learning. Indeed, the difference between those approaches is related to the parameter estimation technique and granular update policies.

In this paper, the adopted policy for both on-line and off-line learning uses ζ_k to update only the granule $i^\star$ with the highest membership degree, i.e.,

$$i^\star = \arg\max_i g_k^i\left(\zeta_k\right). \tag{20}$$

In this sense, the parameter estimation when the on-line learning is activated is performed by means of the RLS algorithm. Therefore, RLS is performed as follows for $l \in \mathbb{N}_{\leq n_x}$.

$$\Upsilon_k = P_k x_k \left(\eta + x_k^\top P_k x_k\right) \tag{21}$$

$$P_{k+1} = \eta^{-1}\left(I_{n_\zeta} - \Upsilon_k x_k^\top\right) P_{k+1} \tag{22}$$

$$\Theta_k^l = \Theta_{k-1}^l + \Upsilon_k\left(x_{k,l} - \zeta_{k-1}^\top \Theta_{k-1}^l\right) \tag{23}$$

where $\Theta_k^l \triangleq \left[[A_k^i]_l \ \ [B_k^i]_l\right]^\top$, with $[A_k^i]_l$ and $[B_k^i]_l$ denoting, respectively, the l-th row of A_k^i and B_k^i, $P_k \in \mathbb{R}^{n_x \times n_x}$ is an estimate of the inverted regularised data autocorrelation matrix, and $\Upsilon_k \in \mathbb{R}^{n_x}$ is the RLS gain vector.

In addition, when a new granule is created, the consequent parameters of the last created granule $\mathcal{G}_k^{N_G}$ are estimated by means of WLS estimation using the data window with the last φ data samples. The WLS is performed as follows for $l \in \mathbb{N}_{\leq n_x}$.

$$\Theta_k^l = \Psi_k^\dagger X_k^l \tag{24}$$

$$\Psi_k = \begin{bmatrix} \zeta_{k-1}^\top \\ \zeta_{k-2}^\top \\ \vdots \\ \zeta_{k-\varphi}^\top \end{bmatrix}, \quad X_k^l = \begin{bmatrix} x_{k,l} \\ x_{k-1,l} \\ \vdots \\ x_{k-\varphi+1,l} \end{bmatrix}$$

where $x_{k,l}$ denotes the l-th element of x_k, and $\Psi^\dagger$ denotes the Moore-Penrose pseudo-inverse of Ψ.

It is worth to notice that the proposed EEFIG learning procedure consists of two main procedures: the increase of the model complexity by creating new granules and the update of model parameters. The update of model parameters consists of estimating the parameters by means of RLS if there was not granule creation. The parameter learning rate by RLS depends on the data stream co-variance and on the forgetting factor η that is chosen to weight new data samples more heavily than the old ones.

Otherwise, the model complexity (number of granules) is controlled by the PJG. In particular, the model complexity is increased only if a data sequence with more than $\hat{n}_a$ is sampled. Since the proposed methodology requires that an offline training is previously performed, the granule creation in the online phase should be infrequent during the online learning phase, although it occurs if the system is exposed to significant dynamic novelties.

3. MPC and MHE based on TS fuzzy models

In this section, the control (MPC) and estimation (MHE) techniques are presented using the TS model formulation introduced in the previous section.

3.1. MPC Design for TS fuzzy models

In this section, it is presented a particular MPC formulation for solving nonlinear control problems based on Takagi-Sugeno fuzzy models for representing the nonlinear dynamics. The proposed MPC approach based on TS fuzzy models is simply called TS-MPC. Hence, in the TS-MPC, the TS fuzzy model is used for predicting the future system behaviour (see Figure 1), including the states and premise variables, and computing optimal control actions.

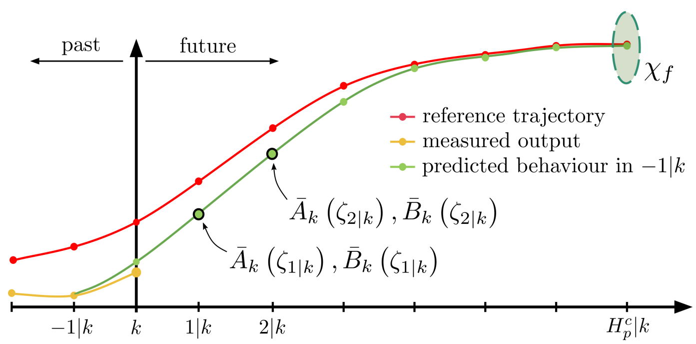


Figure 1: MPC behaviour at time k.

However, computing such a prediction in a particular forward horizon when using a TS fuzzy system can be a challenging task, since such models depend on membership degrees that are functions of the same variables (e.g., states) that are predicted. Thus, sometimes, predicting state variables from TS fuzzy models can lead to errors in the model instantiation due to a lack of knowledge of future behaviour. To overcome this issue, the predicted state and control action vectors in the past optimal realisation are used to obtain the appropriate prediction of premise variables in k as

$$\zeta_{i|k} = \left[x_{i|k-1}^{\top} \; u_{i|k-1}^{\top}\right]^{\top}, \; \forall i = 0, ..., H_p^c \tag{25}$$

and the membership degrees $g_k^i(\zeta_k)$, where $H_p^c \in \mathbb{N}$ is the prediction horizon.

Hence, the use of state space TS models allows to formulate the MPC strategy as a quadratic optimization problem that is solved at each time k to determine the control actions considering that the values of $\hat{x}_k$ and u_{k-1} are known

$$\begin{aligned}
\min_{\Delta\tilde{U}_k,\tilde{X}_k} \quad & \mathrm{J_k} = \sum_{i=0}^{H_p^c-1} \Big((r_{i|k} - x_{i|k})^\top Q_c (r_{i|k} - x_{i|k}) + \Delta u_{i|k} R_c \Delta u_{i|k} \Big) + x_{H_p^c|k}^\top P_c x_{H_p^c|k} \\
\text{s.t.} \quad & \tilde{x}_{i+1|k} = \bar{A}_k\left(\zeta_{i|k}\right)\tilde{x}_{i|k} + \bar{B}_k\left(\zeta_{i|k}\right)u_{i|k} \\
& \tilde{u}_{i|k} = \tilde{u}_{i-1|k} + \Delta\tilde{u}_{i|k} \\
& \tilde{u}_{i|k} \in \Pi \\
& \Delta\tilde{u}_{i|k} \in \Delta\Pi \\
& \tilde{x}_{H_p^c|k} \in \chi_f \\
& \tilde{x}_{0|k} = \hat{x}_k \\
& \tilde{u}_{-1|k} = u_{k-1} \\
& \forall i = 0, ..., H_p^c \ ,
\end{aligned} \tag{26}$$

where $\bar{A}_k\left(\zeta_{i|k}\right)$ and $\bar{B}_k\left(\zeta_{i|k}\right)$ are the state-space TS matrices obtained with (12) and (13), respectively and computed by means of the updated premise variables $\zeta_{i|k}$. $\tilde{x}_{i|k} \in \mathbb{R}^{n_x}$ and $\tilde{u}_{i|k} \in \mathbb{R}^{n_u}$ are the i-steps ahead state and input predictions, respectively. $r_k \in \mathbb{R}^{n_x}$ denotes the reference, $\Delta\tilde{u}_k \in \mathbb{R}^{n_u}$ is the system input variation, $\hat{x}_k \in \mathbb{R}^{n_x}$ is the current estimated state vector. The sets $\Pi = \{u_k | A_u u_k \leq b_u\}$ and $\Delta\Pi = \{\Delta u_k | A_{\Delta u}\Delta u_k \leq b_{\Delta u}\}$ constrain the system inputs and their variations, respectively. The tuning matrices $Q_c \in \mathbb{R}^{n_x\times n_x}$ and $R_c \in \mathbb{R}^{n_u\times n_u}$ are positive definite to ensure the convexity of the cost function $\mathrm{J_k}$.
The closed loop stability is guaranteed by introducing a terminal Lyapunov function $P_c \in \mathbb{R}^{n_x\times n_x} > 0$ and a terminal domain χ_f as constraint. Both are computed following the design procedure presented in the previous work [1]. The vectors of optimal states and control actions are defined as

$$\tilde{X}_k = \begin{bmatrix} \tilde{x}_{1|k} \\ \tilde{x}_{2|k} \\ \vdots \\ \tilde{x}_{H_p^c|k} \end{bmatrix} \in \mathbb{R}^{H_p^c\times n_x} \ , \tilde{U}_k = \begin{bmatrix} \tilde{u}_{0|k} \\ \tilde{u}_{1|k} \\ \vdots \\ \tilde{u}_{H_p^c-1|k} \end{bmatrix} \in \mathbb{R}^{H_p^c\times n_u} \ . \tag{27}$$

Note that the time discretisation is embedded inside the identification procedure such that the learned TS system is already a discrete-time model.

3.2. MHE Design for TS fuzzy models

Most of control applications require of a set of variables from the real plant that most of times either cannot be obtained using sensors or is considerably complicated. In this paper, we solve the estimation problem using the MHE approach based on TS fuzzy models. Then, the proposed MHE approach based on TS fuzzy models is simply called TS-MHE.

The aim of the MHE is to compute the current dynamic states by means of a constrained optimization that uses a set of past data and employing a system model for computing the current state.

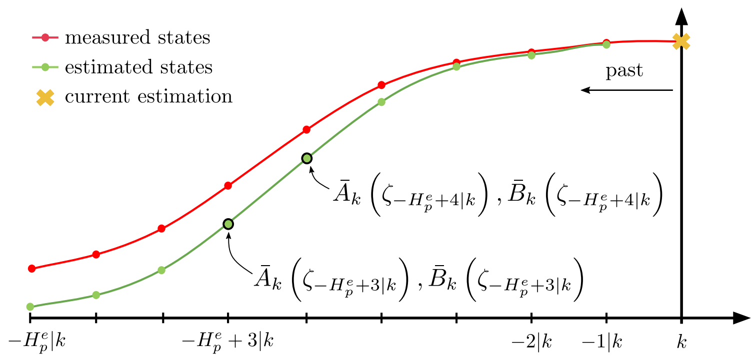


Figure 2: MHE behaviour at time k.

At this point, the set of model instantiations, i.e. $\bar{A}_k(\zeta_{i|k})$ and $\bar{B}_k(\zeta_{i|k})$, depend on a premise vector composed by past estimated states and control input variables

$$\zeta_{i|k} = \left[\hat{x}_{i|k-1}^\top \; u_{i|k-1}^\top\right]^\top \;, \; \forall i = -H_p^e, ..., -1 \tag{28}$$

and the membership degrees $g_k^i\left(\zeta_k\right)$, where $H_p^e \in \mathbb{N}$ is the length of the past estimation window.

Therefore, considering the past estimation $\hat{x}_{-H_p^e|k}$ as the starting state and the known output and input system vectors as

$$Y_k = \begin{bmatrix} y_{-H_p^e|k} \\ y_{-H_p^e+1|k} \\ \vdots \\ y_{-1|k} \end{bmatrix} \in \mathbb{R}^{H_p^e \times n_{xm}} \;, U_{k-1} = \begin{bmatrix} u_{-H_p^e|k-1} \\ u_{-H_p^e+1|k-1} \\ \vdots \\ u_{-1|k-1} \end{bmatrix} \in \mathbb{R}^{H_p^e \times n_u} \;, \tag{29}$$

then, the following quadratic optimization problem can be run at each time k to reduce the measurement noise and estimating those states that are currently unavailable

$$
\begin{aligned}
\min_{\hat{X}_k} \quad & \mathrm{J_k} = \sum_{i=-H_p^e}^{-1} \left(w_{i|k}^{\top} Q_e w_{i|k} + s_{i|k}^{\top} R_e s_{i|k} \right), \\
\text{s.t.} \quad & \hat{x}_{i+1|k} = \bar{A}_k \left(\zeta_{i|k}\right) \hat{x}_{i|k} + \bar{B}_k \left(\zeta_{i|k}\right) u_{i|k} + w_{i|k} \\
& y_{i|k} = C\hat{x}_{i|k} + s_{i|k} \\
& \hat{X}_k \in X_d \\
& \forall i = -H_p^e, ..., -1 \ .
\end{aligned}
\tag{30}
$$

This optimal estimation problem is solved on-line for

$$
\hat{X}_k = \begin{bmatrix} \hat{x}_{-H_p^e+1|k} \\ \hat{x}_{-H_p^e+2|k} \\ \vdots \\ \hat{x}_{0|k} \end{bmatrix} \in \mathbb{R}^{H_p^e \times n_{\hat{x}}} \ , \tag{31}
$$

where $\bar{A}_k \left(\zeta_{i|k}\right)$ and $\bar{B}_k \left(\zeta_{i|k}\right)$ are the state-space TS matrices obtained with (12) and (13), respectively, computed with the updated premise vector defined in (28). $w_{i|k} \in \mathbb{R}^{n_x}$ and $s_{i|k} \in \mathbb{R}^{n_{xm}}$ represent the estimation error and the process noise, respectively, being $n_{\hat{x}}$ and n_{xm} the number of estimated and measured states, respectively. $y_{i|k}$ is the measured output vector and C is the output matrix. $X_d \in \mathbb{R}^{n_{\hat{x}} \times n_{\hat{x}}}$ is the constraint region for the system states and its defined as $X_d = \{\hat{x}_k | A_x \hat{x}_k \leq b_x\}$. Matrices $Q_e = Q_e^{\top} \in \mathbb{R}^{n_{\hat{x}} \times n_{\hat{x}}}$ and $R_e = R_e^{\top} \in \mathbb{R}^{n_{xm} \times n_{xm}}$, are positive definite to generate a convex cost function. Note that, unlike MPC technique, MHE strategy performs an optimization taken into account a window of past data.

4. Integrating the MPC and MHE designs on a TS on-line fuzzy learning

In this section, the overall integration of MPC, MHE and online learning of a TS-EEFIG model is described. In Section 2, the TS-EEFIG model is explained as well as the TS-EEFIG learning procedures, i.e., the update of granules and the estimation of their consequent parameters. Algorithm 1 summarises the main steps of the on-line TS-EEFIG learning procedure. The Algorithm 1 is executed at each time step to provide learning abilities to the MPC and MHE framework.

It is worth mentioning that the TS-EEFIG model can be also obtained by means of batch data, i.e. using off-line learning. The general procedures presented in Algorithm 1 are also valid for off-line learning, since the batch data can be presented one sample at a time. However, for off-line learning the consequent parameter estimation is always performed by means of WLS by using (24). It is recommended that the

Algorithm 1: Online Granular TS Model Learning Procedure .

Data: New data sample ζ_k and TS-EEFIG with granules $\mathcal{G}^i_{k-1}$ and parameters A^i_{k-1} and B^i_{k-1} for $i \in \mathbb{N}_{\leq N_G}$
Result: Updated granules $\mathcal{G}^i_k$ and parameters A^i_k and B^i_k for $i \in \mathbb{N}_{\leq N_G}$
if $k > \varphi$ **then**
  Update the Σ^{aux}_k and ν^{aux}_k using (18) and (15) ;
  for $i = 1$ **to** N_G **do**
    if $\zeta_k \in \mathcal{E}^i_{k-1}$ **then**
      `anomaly` = `FALSE` ;
      Compute ν^i_k, $L^i_{k,j}$, $R^i_{k,j}$, Σ^i_k, $\underline{\zeta}^i_k$, and $\overline{\zeta}^i_k$ by means of (15)–(18) and (6)–(7) ;
      if $\mathcal{I}\left(\mathcal{G}^i_k, \zeta_k\right) < \mathcal{I}\left(\mathcal{G}^i_{k-1}, \zeta_k\right)$ **then**
        $\nu^i_k = \nu^i_{k-1}$, , $L^i_{k,j} = L^i_{k-1,j}$, $R^i_{k,j} = R^i_{k-1,j}$, $\Sigma^i_k = \Sigma^i_{k-1}$, $\underline{\zeta}^i_k = \underline{\zeta}^i_{k-1}$, and $\overline{\zeta}^i_k = \overline{\zeta}^i_{k-1}$;
      end if
    else
      $\nu_k = \nu_{k-1}$, , $L^i_{k,j} = L^i_{k-1,j}$, $R^i_{k,j} = R^i_{k-1,j}$, $\Sigma^i_k = \Sigma^i_{k-1}$, $\underline{\zeta}^i_k = \underline{\zeta}^i_{k-1}$, and $\overline{\zeta}^i_k = \overline{\zeta}^i_{k-1}$;
    end if
  end for
  if `anomaly` **then**
    Increment n_a;
    if $n_a > \hat{n}_a$ **then**
      Increment N_G;
      Create $\mathcal{G}^{N_G}_k$ with $\nu^{N_G}_k$, $L^{N_G}_{k,j}$, $R^{N_G}_{k,j}$, $\Sigma^{N_G}_k$, $\underline{\zeta}^{N_G}_k$, and $\overline{\zeta}^{N_G}_k$ computed by means of $\zeta_k, \ldots, \zeta_{k-\varphi}$;
      Estimate $A^{N_G}_k$ and $B^{N_G}_k$ by means of WLS using (24) ;
    else
      Estimate $A^{i^\star}_k$ and $B^{i^\star}_k$ by means of RLS using (21),(22) and (23) for $i^\star$ given by (20);
    end if
  else
    $n_a = 0$;
    Estimate $A^{i^\star}_k$ and $B^{i^\star}_k$ by means of RLS using (21),(22) and (23) for $i^\star$ given by (20);
  end if
else
  Save ζ_k in the buffer ;
  Update the Σ^{aux}_k and ν^{aux}_k using (18) and (15) ;
  $\mathcal{G}^i_k \leftarrow \mathcal{G}^i_{k-1}$, $A^i_k = A^i_{k-1}$, and $B^i_k = B^i_{k-1}$ $\forall i \in \mathbb{N}_{\leq N_G}$;
end if

data used for obtaining the initial model are persistently exciting in order to avoid excessive performance degradation at the first instants of the online usage of that model.

The complete workflow scheme used recursively at every sampling period is explained hereafter in detail and graphically represented in Figure 3.

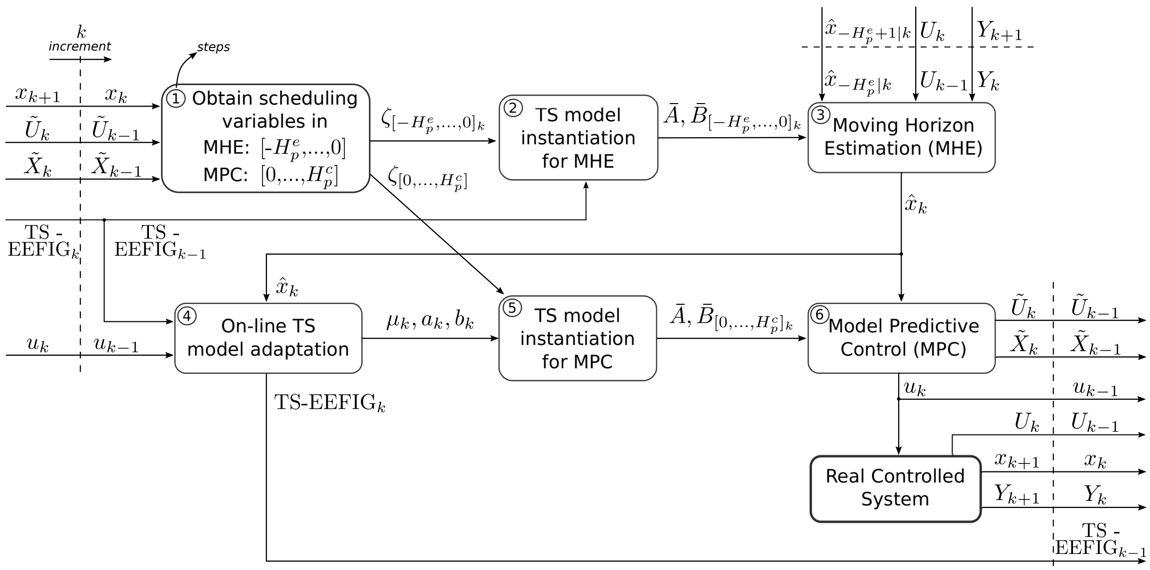


Figure 3: Workflow diagram for k iteration.

- **Step 1**: At the beginning of every iteration k, it is necessary to compute the previous membership degree by means of the vector premise variables ζ_{k-1} that concatenates both x_{k-1} and u_{k-1}. That vector is updated and ζ_k is obtained throughout the control and estimation windows (see Figures 1 and 2). On one hand, for MPC control purposes a prediction of future premise variables is required. For this purpose, the optimal prediction set of inputs and states in last iteration is used to achieve an admissible guess of the future values of the vector of premise variables, i.e. $\zeta_{i|k}, \forall i = 0, ..., H_p^c$. On the other hand, for the MHE procedure, a particular window of past states and inputs is used to set up the past values of the premise vector, i.e. $\zeta_{i|k},\ \forall i = -H_p^e, ..., -1$.

- **Step 2**: The TS model instantiation for estimation is performed. To do so, equations (9)-(13) are used to obtain the TS state-space matrices $\bar{A}_k\left(\zeta_{i|k}\right)$ and $\bar{B}_k\left(\zeta_{i|k}\right)$ for each index i-step within the estimation window.

- **Step 3**: At this point, the MHE technique uses the computed TS model matrices, the initial state for estimation $x_{-H_p^e|k}$ and the known output (Y_k) and input (U_k) system vectors to compute the next estimated state $(\hat{x}_k)$ by solving the problem (30). Note that, vectors U_k and Y_k are buffers of past control actions applied to the vehicle and measured state variables, respectively.

- **Step 4**: In this step, the on-line learning approach presented in Section (2) is used to update the current TS model with new model dynamics. This approach needs the complete state vector what directly leads to use the current estimated vector $(\hat{x}_k)$ and input vector to the system (u_k), i.e., the premise variable vectors for the TS-EEFIG is defined as $\zeta_k \triangleq \left[\hat{x}_k^\top \; u_k^\top\right]^\top$. Therefore, the previous TS-EEFIG model with granules $\mathcal{G}_{k-1}^i$ and parameters A_{k-1}^i and B_{k-1}^i for $i \in \mathbb{N}_{\leq N_G}$ and ζ_k are used as input in Algorithm 1 to compute the updated TS-EEFIG model with granules $\mathcal{G}_k^i$ and parameters A_k^i and B_k^i.

- **Step 5**: The TS model instantiation for the MPC technique is performed. For this purpose, as in step 2, (12) and (13) are used to obtain the TS state-space matrices for each i-steps ahead within the prediction window of length H_p^c.

- **Step 6**: Finally, the constrained optimal control problem in (26) is solved to compute the next optimal control action (u_k) that will lead the real system to the reference tracking.

5. Application to Autonomous Driving

In this section, the TS-EEFIG learning model is used to enhance control and estimation capabilities in autonomous driving, in particular in racing applications. The scheme presented in Figure 4 represents the proposed autonomous driving control framework that includes trajectory planning, control and estimation, that provided by MPC and MHE based on the TS fuzzy models (TS-MPC and TS-MHE).

5.1. Vehicle description

The platform used for testing is a 1/10 Scale RC car is already used in [2] and is shown in Figure 5. This vehicle includes a basic net of sensors that allows to measure most of the needed variables for control. The vehicle is actuated by varying the quantity of linear acceleration in the rear wheel and the steering angle at the front wheel to produce motion. We use a high fidelity vehicle bicycle model for simulation. The set of differential equations defining its physical behavior are the following

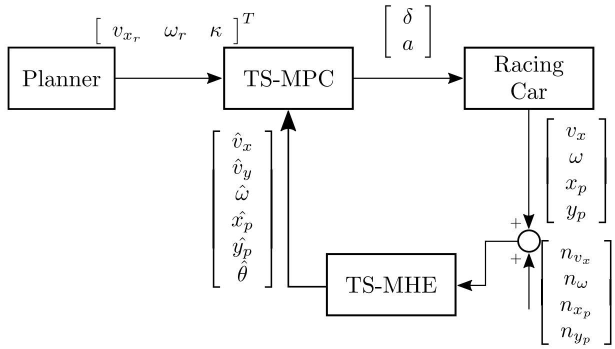


Figure 4: Planning-Control-Estimation diagram for autonomous driving.

Figure 5: Picture of the scale vehicle.

$$
\begin{aligned}
\dot{x_p} &= \cos\theta v_x - \sin\theta v_y \\
\dot{y_p} &= \sin\theta v_x + \cos\theta v_y \\
\dot{\theta} &= \omega \\
\dot{v}_x &= a - \frac{F_{yF}\sin\delta}{m} - \mu g + \omega v_y \\
\dot{v}_y &= \frac{F_{yF}\cos\delta}{m} + \frac{F_{yR}}{m} - \omega v_x \\
\dot{\omega} &= \frac{F_{yF} l_f \cos\delta - F_{yR} l_r}{I} \\
F_{yF} &= D_p \sin\left(C_p \tan^{-1}(B_p \alpha_f)\right) \\
F_{yR} &= D_p \sin\left(C_p \tan^{-1}(B_p \alpha_r)\right) \\
\alpha_f &= \delta - \tan^{-1}\left(\frac{v_y}{v_x} - \frac{l_f \omega}{v_x}\right) \\
\alpha_r &= -\tan^{-1}\left(\frac{v_y}{v_x} + \frac{l_r \omega}{v_x}\right)
\end{aligned}
\tag{32}
$$

where the dynamic vehicle variables v_x, v_y and ω represent the body frame velocities, i.e. linear velocity in x, linear velocity in y and angular rate around the orthogonal axis to the road, respectively. Variables x_p, y_p and θ are the position coordinates and orientation, respectively, both with respect to a global frame. The control variables δ and a are the steering angle at the front wheels and the longitudinal acceleration vector on the rear wheels, respectively. Front and rear slip angles are represented as α_f and α_r, respectively. F_{yf} and F_{yr} are the lateral forces produced in front and rear tires, respectively. B_p, C_p and D_p define the shape of the Pacejka"Magic Formula" tire model. m and I represent the vehicle mass and inertia and l_f and l_r are the distances from the vehicle center of mass to the front and rear wheel axes, respectively. μ, ρ and g are the static friction coefficient and the earth gravity, respectively. All parameters are properly defined in Table 1. In addition, with the

Table 1: Vehicle parameters

Parameter	Value	Parameter	Value
l_f	0.125 m	l_r	0.125 m
m	1.98 kg	I	0.03 kg m^2
D_p	7.76	C_p	1.6
B_p	6	μ	0.1

aim of adding more realistic conditions to the simulation problem, white Gaussian noise magnitudes are added to measured states with zero mean and covariances

$$Co_{v_x} = 1 \times 10^{-3}\ ,\ Co_{\omega} = 4 \times 10^{-3}\ ,\ Co_x = 5 \times 10^{-3}\ ,\ Co_y = 5 \times 10^{-3}\ . \tag{33}$$

5.2. TS model instantiation

Both MPC and MHE approaches based on TS fuzzy models require proper TS instantiations of the model of the system to perform the appropriate controls and estimations. Consequently, these instantiations depend on a changing premise vector defined as

$$\zeta_k := \left[\begin{array}{ccccc} \hat{v_x} & \hat{v_y} & \hat{\omega} & \delta & a \end{array}\right]^\top . \tag{34}$$

The determination of past premise variables to instantiate the model for estimation is not complicated. This consists on storing past variables in a vector of length H_p^e. However, obtaining the set of future premise variables to instantiate the TS model for the MPC prediction stage is not a trivial task. In this particular driving application, the predicted data obtained from the past optimal procedure ($\tilde{X}_{k-1}$ and $\tilde{U}_{k-1}$) as well as the references coming from the planning stage are used as information source to set the future premise vector (ζ_k) and compute the next set of $\bar{A}_k(\zeta_{i|k})$ and $\bar{B}_k(\zeta_{i|k})$ instantiations for each one of the i-steps in the prediction stage using (9)-(13).

5.3. Control and estimation set up

For the MPC problem (26), the state, input and reference vectors are defined as follows

$$x = [v_x \; v_y \; \omega \; e_y \; \theta_e]^\top \,, \;\; u = [\delta \; a]^\top \,, \;\; r = [v_{x_r} \; 0 \; \omega_r \; 0 \; 0]^\top \tag{35}$$

where v_{x_r} and ω_r are the longitudinal velocity and angular references, respectively. States e_y and θ_e are the lateral and orientation errors, respectively, with respect to the road center line and are computed as

$$\begin{aligned} \dot{y}_e &= v_x \sin\theta_e + v_y \cos\theta_e \\ \dot{\theta}_e &= \omega - \frac{v_x \cos\theta_e - v_y \sin\theta_e}{1 - y_e \kappa} \kappa \;. \end{aligned} \tag{36}$$

where κ represents the road curvature and is provided by the trajectory planner. The proposed MPC approach is performed with sampling time of 20 ms. The tuning aims to minimise the longitudinal velocity and lateral position errors while computing smooth control actions. The diagonal terms of the weighting matrices in the cost function and prediction horizon of MPC problem (26), found by iterative tuning until the desired performance is achieved, are

$$\begin{aligned} Q_c &= 0.8 \; \mathrm{diag}\{ \; 0.25 \quad 10^{-6} \quad 10^{-6} \quad 0.7 \quad 0.07 \; \} \\ R_c &= 0.2 \; \mathrm{diag}\{ \; 1 \quad 0.06 \; \} \\ H_p^c &= 10 \;. \end{aligned} \tag{37}$$

The MPC input constraints sets Π and $\Delta\Pi$ are defined as

$$A_u = \begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 0 & 1 \\ 0 & -1 \end{bmatrix}, \quad b_u = \begin{bmatrix} 0.4 \\ 0.4 \\ 2 \\ 1 \end{bmatrix}, \tag{38}$$

$$A_{\Delta u} = \begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 0 & 1 \\ 0 & -1 \end{bmatrix}, \quad b_{\Delta u} = \begin{bmatrix} 0.05 \\ 0.05 \\ 0.5 \\ 0.5 \end{bmatrix} \;. \tag{39}$$

Regarding to the MHE problem (30), the state, input and output system vectors are

$$x = [v_x \; v_y \; \omega \; x_p \; y_p \; \theta]^\top \,, \;\; u = [\delta \; a]^\top \;\; , y = [v_x \; \omega \; x_p \; y_p]^\top \,. \tag{40}$$

The tuning aims to minimise the process noise while computing the right value of v_y and θ by using the TS model representation of the vehicle. The diagonal terms of the weighting matrices in the cost function and past horizon of problem (30), found by iterative tuning until the desired performance is achieved, are

$$\begin{aligned} Q_e &= \mathrm{diag}\{\ 0.05 \quad 0.03 \quad 0.03 \quad 0.0025 \quad 0.0025 \quad 0.0025\ \}, \\ R_e &= \mathrm{diag}\{\ 0.01 \quad 0.02 \quad 0.01 \quad 0.01\ \}, \\ H_p^e &= 7\ , \end{aligned} \tag{41}$$

where both matrices contain in their diagonal the variance of each state variable and measured variable, respectively. The TS-MHE state region is defined in the polytope defined by

$$A_x = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & -1 \end{bmatrix}, \quad b_x = \begin{bmatrix} 5 \\ -0.1 \\ 0.5 \\ 0.5 \\ 5 \\ 5 \\ \infty \\ \infty \\ \infty \\ \infty \\ \infty \\ \infty \end{bmatrix} \tag{42}$$

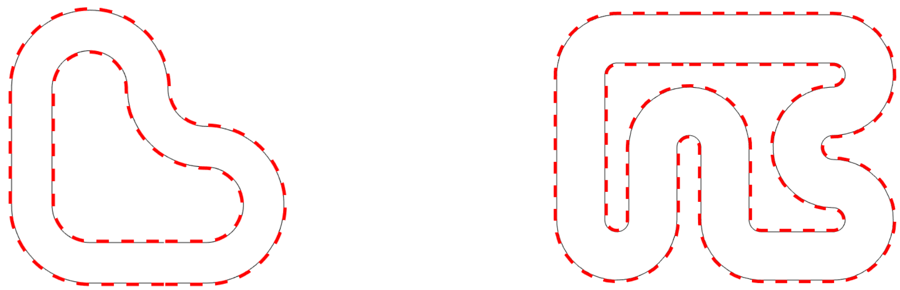
and the output matrix is defined as

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}, \tag{43}$$

corresponding with the available state variables for measurement (v_x, ω, x_p and y_p).

5.4. Results

In this section, we present the evaluation of the proposed MPC and MHE methodology using the TS-EEFIG models in both ways: the off-line modeling from batch data is obtained in racing scenario but without online learning; and the TS-EEFIG is obtained off-line but improved by on-line learning when it is used in different challenging tracks for racing driving.



(a) Track 1 (b) Track 2

Figure 6: Tracks used for simulations.

The experiments consists on learning the vehicle dynamics also in an off-line way using the TS-EEFIG approach in Track 2 (see Figure 6b). The off-line learned model is used combined with the proposed tracking MPC and MHE framework presented in Section 3.1 in the same Track 2 in order to validate the model. In sequel, this model is used in Track 1 (see Figure 6a) where it continues learning and is improved when the proposed MPC technique based on TS-EEFIG model is performed.
Both TS-MPC and TS-MHE algorithms are coded in MATLAB framework. Yalmip [32] and GUROBI [33] are used for solving both quadratic optimization problems and they run on a DELL inspiron 15 (Intel core i7-8550U CPU @ 1.80GHzx8).

Firstly, the vehicle dynamic model is learned in Track 2 (see Figure 6b) using the TS approach presented in Section 2. Such a dynamic identification is made off-line and the trajectory tracking performance when is coupled with the MPC control technique is presented in Figure 7. Track 2 is much more challenging than Track 1 and this is shown in Figure 8 where it is shown the temporal evolution of some vehicle variables during two laps. We can see wider ranges for control input variables (δ and a) but also more changing linear and angular velocities which lead to a complete control challenge.

The results in Figures 7 and 8 indicate that the TS-EEFIG model learned offline is able to represent adequately the system dynamics and the proposed MPC ensures the reference tracking.

Secondly, the on-line learning technique is tested in Track 1. For that purpose, we hold the TS-EEFIG model learned for Track 2 as a basis model and start tracking the reference proposed for Track 1 using the MPC technique. Note that, no previous knowledge about Track 1 is used. At every control iteration k, the on-line learning strategy presented in Section 2 and summarised by Algorithm 1 takes the last applied inputs (δ_{k-1} and a_{k-1}) and the estimated state vector ($\hat{x}_k$) to update the current TS model (TS-EEFIG$_k$).

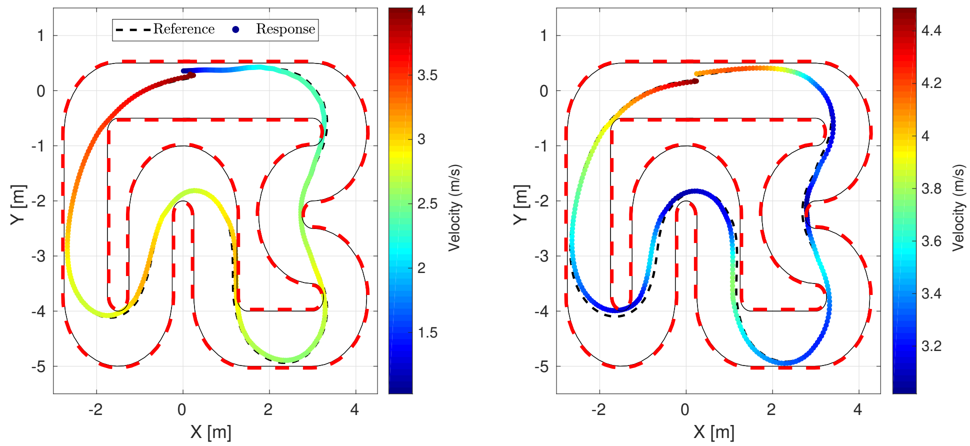


Figure 7: Control tracking response in terms of vehicle speed using the EEFIG algorithm without online learning in Track 2. Left: first lap. Right: second lap.

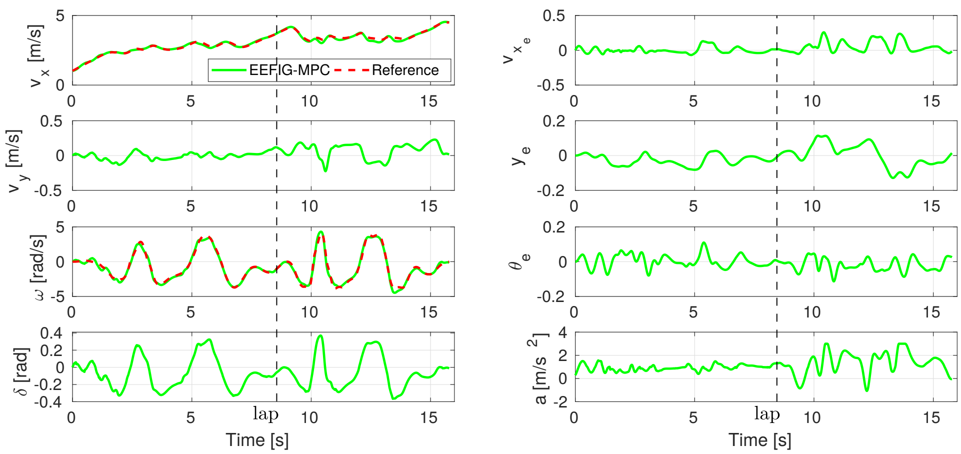


Figure 8: Time evolution of vehicle dynamic variables (v_x, v_y, ω), controlled variables (v_{x_e}, y_e, θ_e) and control actions (δ, a) throughout the simulation in Track 2.

In Figure 9, three consecutive laps are depicted. During the first lap, the vehicle is accelerating and still not able to track perfectly the reference. However, during second and third laps the on-line learning algorithm updates properly the initial TS model to finally track the proposed reference. The results indicates the potential of the proposed data-driven MPC based on the learned TE-EEFIG models for adapting the learned dynamics to unknown driving circumstances. Furthermore, note in

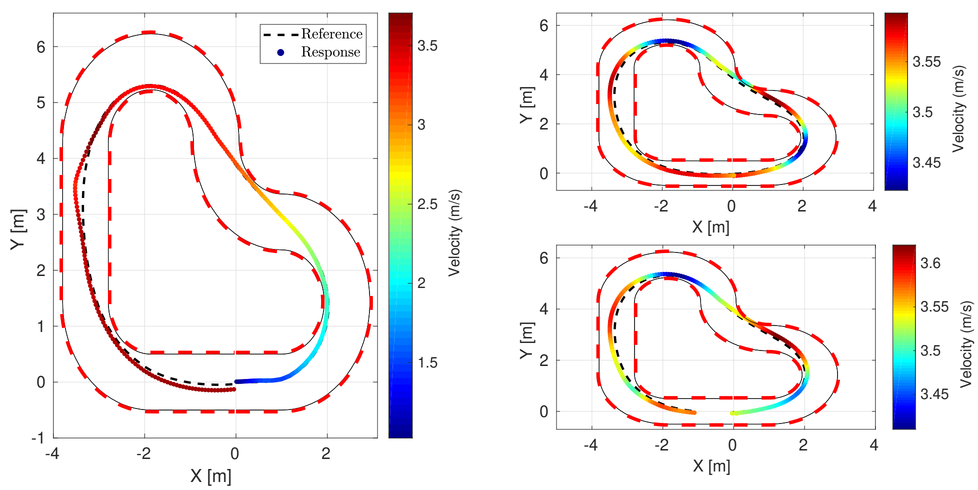


Figure 9: Control tracking response using the on-line EEFIG adaptation algorithm in Track 1. Left: first lap. Upper right: second lap. Lower right: third lap.

Figure 10 that, after the first lap, the velocity references are tracked with slightly low error and the closed loop system is stable in this challenging driving situation showing then the impressive capabilities of the proposed technique.

6. Conclusions

In this paper, an on-line learning approach to obtain and update a TS formulation of the vehicle dynamic model has been proposed. The learned TS model has been oriented to enhance model-based control and estimation techniques. In particular, the Evolving Ellipsoidal Fuzzy Information Granules (EEFIG) method is used to obtain polytopic representations by means of ellipsoidal membership degrees for nonlinear multidimensional systems This approach is based on granular computing and is able to improve itself when it is presented to new dynamically significant data. Finally, both TS-MPC and TS-MHE strategies have been proposed as techniques to solve complex control and estimation problems in real-time. Both approaches have

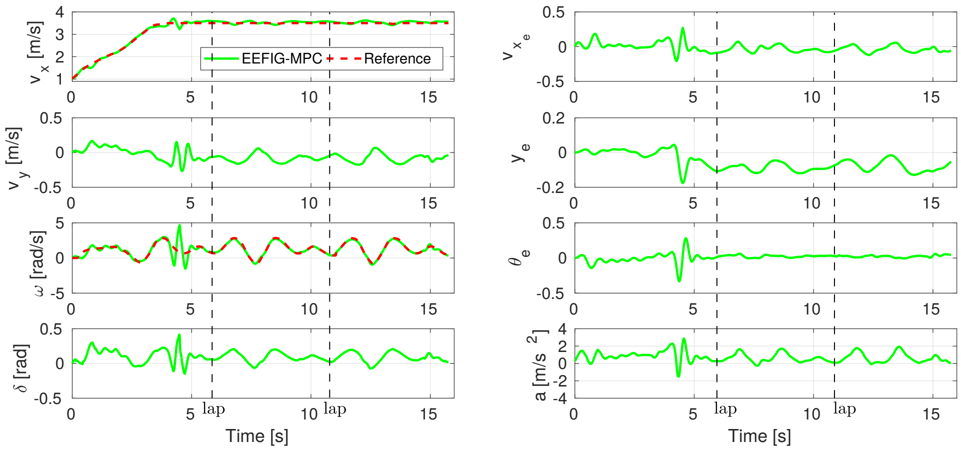


Figure 10: Time evolution of vehicle dynamic variables (v_x, v_y, ω), controlled variables (v_{x_e}, y_e, θ_e) and control actions (δ, a) throughout the simulation in Track 1.

been tested in challenging scenarios where a vehicle drive autonomously in racing mode under realistic conditions. The results show the high potential of the proposed learning approach adapting the vehicle TS model to any scenario and the promising capabilities of MPC and MHE based on TS-EEFIG learning models in optimal control and optimal estimation achieving a low computational effort. As future work, we propose to deal with the whole nonlinear dynamics of autonomous vehicles by using evolving TS models to test the proposed approach in a real scale vehicle.

Acknowledgements

This work has been funded by the Spanish Ministry of Economy and Competitiveness (MINECO) and FEDER through the project SCAV (ref. DPI2017-88403-R). The authors are supported by a FI AGAUR grant (ref 2017 FI B00433) and the Brazilian agency CNPq under grants 307933/2018-0 and 201370/2019-0.

References

[1] E. Alcalá, V. Puig, J. Quevedo, TS-MPC for autonomous vehicles including a TS-MHE-UIO estimator, IEEE Transactions on Vehicular Technology 68 (7) (2019) 6403–6413.

[2] E. Alcalá, V. Puig, J. Quevedo, U. Rosolia, Autonomous racing using linear

parameter varying-model predictive control, Control Engineering Practice (95) (2020) 104270.

[3] D. Bicego, J. Mazzetto, R. Carli, M. Farina, A. Franchi, Nonlinear model predictive control with enhanced actuator model for multi-rotor aerial vehicles with generic designs, Journal of Intelligent & Robotic Systems 100 (3-4) (2020) 1213–1247.

[4] U. Rosolia, X. Zhang, F. Borrelli, Data-driven predictive control for autonomous systems, Annual Review of Control, Robotics, and Autonomous Systems 1 (1) (2018) 259–286.

[5] S. Givigi, P. T. Jardine, Machine learning for data-driven control of robots, IEEE Potentials 37 (4) (2018) 35–39.

[6] Z. Wu, A. Tran, D. Rincon, P. D. Christofides, Machine learning-based predictive control of nonlinear processes. part i: Theory, AIChE Journal 65 (11) (2019).

[7] Z. Wu, A. Tran, D. Rincon, P. D. Christofides, Machine-learning-based predictive control of nonlinear processes. part ii: Computational implementation, AIChE Journal 65 (11) (2019).

[8] Z. Wu, D. Rincon, P. D. Christofides, Real-time adaptive machine-learning-based predictive control of nonlinear processes, Industrial & Engineering Chemistry Research 59 (6) (2020) 2275–2290.

[9] A. E. Sallab, M. Abdou, E. Perot, S. Yogamani, Deep reinforcement learning framework for autonomous driving, Electronic Imaging 2017 (19) (2017) 70–76.

[10] S. Lefèvre, A. Carvalho, F. Borrelli, A learning-based framework for velocity control in autonomous driving, IEEE Transactions on Automation Science and Engineering 13 (1) (2015) 32–42.

[11] S. Lefèvre, A. Carvalho, F. Borrelli, Autonomous car following: A learning-based approach, in: 2015 IEEE Intelligent Vehicles Symposium (IV), IEEE, 2015, pp. 920–926.

[12] J. Kabzan, L. Hewing, A. Liniger, M. N. Zeilinger, Learning-based model predictive control for autonomous racing, IEEE Robotics and Automation Letters 4 (4) (2019) 3363–3370.

[13] U. Rosolia, F. Borrelli, Learning model predictive control for iterative tasks. a data-driven control framework, IEEE Transactions on Automatic Control 63 (7) (2017) 1883–1896.

[14] U. Rosolia, A. Carvalho, F. Borrelli, Autonomous racing using learning model predictive control, in: 2017 American Control Conference (ACC), IEEE, 2017, pp. 5115–5120.

[15] U. Rosolia, F. Borrelli, Learning how to autonomously race a car: A predictive control approach, IEEE Transactions on Control Systems Technology 28 (6) (2020) 2713–2719.

[16] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, E. A. Theodorou, Information-theoretic model predictive control: Theory and applications to autonomous driving, IEEE Transactions on Robotics 34 (6) (2018) 1603–1622.

[17] M. M. Ferdaus, M. Pratama, S. G. Anavatti, M. A. Garratt, E. Lughofer, PAC: a novel self-adaptive neuro-fuzzy controller for micro aerial vehicles, Information Sciences 512 (2020) 481 – 505.

[18] L. A. Q. Cordovil, P. H. S. Coutinho, I. Bessa, M. F. S. V. D'Angelo, R. M. Palhares, Uncertain data modeling based on evolving ellipsoidal fuzzy information granules, IEEE Transactions on Fuzzy Systems 28 (10) (2020) 2427–2436.

[19] P. P. Angelov, X. Gu, Autonomous Learning Multi-model Systems, Springer International Publishing, Cham, 2019, pp. 199–222.

[20] E. Lughofer, M. Pratama, I. Škrjanc, Incremental rule splitting in generalized evolving fuzzy systems for autonomous drift compensation, IEEE Transactions on Fuzzy Systems 26 (4) (2018) 1854–1865.

[21] E. A. Jaleel, K. Aparna, Identification of realistic distillation column using hybrid particle swarm optimization and narx based artificial neural network, Evolving Systems 10 (2) (2019) 149–166.

[22] C. Garcia, A. Esmin, D. Leite, I. Škrjanc, Evolvable fuzzy systems from data streams with missing values: With application to temporal pattern recognition and cryptocurrency prediction, Pattern Recognition Letters 128 (2019) 278 – 282.

[23] C. Luo, H. Wang, Fuzzy forecasting for long-term time series based on time-variant fuzzy information granules, Applied Soft Computing 88 (2020) 106046.

[24] M. Pratama, W. Pedrycz, E. Lughofer, Evolving ensemble fuzzy classifier, IEEE Transactions on Fuzzy Systems 26 (5) (2018) 2552–2567.

[25] M. Inacio, A. Lemos, W. Caminhas, Fault diagnosis with evolving fuzzy classifier based on clustering algorithm and drift detection, Mathematical Problems in Engineering 2015 (2015) 1–14.

[26] M. O. Camargos, I. Bessa, M. F. S. V. D'Angelo, L. B. Cosme, R. M. Palhares, Data-driven prognostics of rolling element bearings using a novel error based evolving Takagi–Sugeno fuzzy model, Applied Soft Computing 96 (2020) 106628.

[27] D. Leite, R. M. Palhares, V. C. S. Campos, F. Gomide, Evolving granular fuzzy model-based control of nonlinear dynamic systems, IEEE Transactions on Fuzzy Systems 23 (4) (2015) 923–938.

[28] E. Alcalá, O. Sename, V. Puig, J. Quevedo, TS-MPC for autonomous vehicle using a learning approach, IFAC-PapersOnLine 53 (2) (2020) 15110–15115, 21th IFAC World Congress.

[29] A. Pedrycz, K. Hirota, W. Pedrycz, F. Dong, Granular representation and granular computing with fuzzy sets, Fuzzy Sets and Systems 203 (2012) 17 – 32.

[30] G. Wang, P. Shi, B. Wang, J. Zhang, Fuzzy n-ellipsoid numbers and representations of uncertain multichannel digital information, IEEE Transactions on Fuzzy Systems 22 (5) (2014) 1113–1126.

[31] S. Dasgupta, Learning mixtures of Gaussians, in: 40th Annual Symposium on Foundations of Computer Science, IEEE, 1999, pp. 634–644.

[32] J. Löfberg, YALMIP: a toolbox for modeling and optimization in MATLAB, in: Computer-Aided Control System Design Conference, Taipei, Taiwan, 2004.

[33] I. Gurobi Optimization, Gurobi optimizer reference manual, URL http://www. gurobi. com (2015).